

Day 24: ユーザー一覧（管理者用）を作ろう

今日のゴール

管理者がユーザーを管理できる画面を実装します。ユーザー一覧表示、ロール変更、アカウント有効化/無効化ができるようにします。

【スクリーンショット: ユーザー管理画面】

なぜこれを作るのか？

チームメンバーを管理する管理機能です。ユーザー管理は学校の出席簿のようなもの。先生が生徒の名前や出席状況を管理するように、管理者がユーザーのアカウント情報やアクセス権限を管理します。

実装ステップ一覧

ステップ	作業内容	所要時間
Step 1	管理者チェック	10分
Step 2	ユーザー一覧表示	15分
Step 3	ロール変更機能	15分
Step 4	アカウント無効化	10分

合計時間: 約50分

Step 1: 管理者チェック (10分)

実装:

```
// filepath: src/app/users/page.tsx
'use client';

import { useSession } from 'next-auth/react';
import { Box, Typography, Alert } from '@mui/material';

export default function UsersPage() {
  const { data: session } = useSession();

  if (session?.user?.role !== 'ADMIN') {
    return (
      <Box sx={{ p: 3 }}>
        <Alert severity="error">
          この画面を表示する権限がありません
        </Alert>
      </Box>
    );
  }

  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4">ユーザー管理</Typography>
    </Box>
  );
}
```

確認ポイント: 管理者のみアクセスできる

Step 2: ユーザー一覧表示 (15分)

実装:

```
// filepath: src/app/users/page.tsx
import { api } from '@trpc/react';
import {
  Table,
  TableBody,
  TableCell,
  TableContainer,
  TableHead,
  TableRow,
  Paper,
  Chip,
  CircularProgress,
} from '@mui/material';

export default function UsersPage() {
  const { data: session } = useSession();
  const { data: users, isLoading } = api.user.getAll.useQuery();

  if (session?.user?.role !== 'ADMIN') {
    return <Alert severity="error">権限がありません</Alert>;
  }

  if (isLoading) {
    return (
      <Box sx={{ display: 'flex', justifyContent: 'center' }}>
        <CircularProgress />
      </Box>
    );
  }

  return (
    <Box sx={{ p: 3 }}>
      <Typography variant="h4" sx={{ mb: 3 }}>ユーザー管理</Typography>

      <TableContainer component={Paper}>
        <Table>

```

```

<TableHead>
  <TableRow>
    <TableCell>名前</TableCell>
    <TableCell>メールアドレス</TableCell>
    <TableCell>ロール</TableCell>
    <TableCell>ステータス</TableCell>
    <TableCell>登録日</TableCell>
  </TableRow>
</TableHead>
<TableBody>
  {users?.map((user) => (
    <TableRow key={user.id}>
      <TableCell>{user.name}</TableCell>
      <TableCell>{user.email}</TableCell>
      <TableCell>
        <Chip
          label={user.role}
          color={user.role === 'ADMIN' ? 'primary' : 'default'}
          size="small"
        />
      </TableCell>
      <TableCell>
        <Chip
          label={user.isActive ? '有効' : '無効'}
          color={user.isActive ? 'success' : 'default'}
          size="small"
        />
      </TableCell>
      <TableCell>
        {new Date(user.createdAt).toLocaleDateString()}
      </TableCell>
    </TableRow>
  ))}
</TableBody>
</Table>
</TableContainer>
</Box>

```

```
);  
}
```

確認ポイント: ユーザー一覧がテーブル形式で表示される

Step 3: ロール変更機能（15分）

実装:

```

// filepath: src/app/users/page.tsx
import { Select, MenuItem } from '@mui/material';

export default function UsersPage() {
  const utils = api.useUtils();

  const updateRoleMutation = api.user.updateRole.useMutation({
    onSuccess: () => {
      utils.user.getAll.invalidate();
    },
  });

  const handleRoleChange = (userId: string, newRole: string) => {
    updateRoleMutation.mutate({
      id: userId,
      role: newRole as 'USER' | 'ADMIN',
    });
  };

  return (
    <TableBody>
      {users?.map((user) => (
        <TableRow key={user.id}>
          <TableCell>{user.name}</TableCell>
          <TableCell>{user.email}</TableCell>
          <TableCell>
            <Select
              size="small"
              value={user.role}
              onChange={(e) => handleRoleChange(user.id, e.target.value)}
              disabled={user.id === session?.user?.id}
            >
              <MenuItem value="USER">USER</MenuItem>
              <MenuItem value="ADMIN">ADMIN</MenuItem>
            </Select>
          </TableCell>
        </TableRow>
      )
    )}
    </TableBody>
  );
}

```

```
        <TableCell>
          <Chip
            label={user.isActive ? '有効' : '無効'}
            color={user.isActive ? 'success' : 'default'}
            size="small"
          />
        </TableCell>
        <TableCell>
          {new Date(user.createdAt).toLocaleDateString()}
        </TableCell>
      </TableRow>
    )}}
  </TableBody>
);
}
```

確認ポイント: ロールをドロップダウンで変更できる

Step 4: アカウト無効化 (10分)

実装:


```

// filepath: src/app/users/page.tsx
import { Switch } from '@mui/material';

export default function UsersPage() {
  const toggleActiveMutation = api.user.toggleActive.useMutation({
    onSuccess: () => {
      utils.user.getAll.invalidate();
    },
  });

  const handleToggleActive = (userId: string, currentStatus: boolean) => {
    toggleActiveMutation.mutate({
      id: userId,
      isActive: !currentStatus,
    });
  };

  return (
    <TableBody>
      {users?.map((user) => (
        <TableRow key={user.id}>
          <TableCell>{user.name}</TableCell>
          <TableCell>{user.email}</TableCell>
          <TableCell>
            {/* ロール選択 */}
          </TableCell>
          <TableCell>
            <Switch
              checked={user.isActive}
              onChange={() => handleToggleActive(user.id, user.isActive)}
              disabled={user.id === session?.user?.id}
            />
            <Chip
              label={user.isActive ? '有効' : '無効'}
              color={user.isActive ? 'success' : 'default'}
              size="small"
            />
          </TableCell>
        </TableRow>
      )
    )}
    </TableBody>
  );
}

```

```
        />
      </TableCell>
      <TableCell>
        {new Date(user.createdAt).toLocaleDateString()}
      </TableCell>
    </TableRow>
  )})
</TableBody>
);
}
```

確認ポイント: スイッチでアカウントを有効/無効化できる

学んだこと

- `session.user.role`: ユーザーのロールで権限チェック
- `Alert` コンポーネント: エラーメッセージ表示
- `Select in Table`: テーブル内でドロップダウン使用
- `Switch` コンポーネント: トグルスイッチUI
- `disabled={user.id === session?.user?.id}`: 自分自身は変更不可

今日のまとめ

- ☐ 管理者チェックを実装できた
- ☐ ユーザー一覧を表示できた
- ☐ ロール変更機能を実装できた
- ☐ アカウント有効化/無効化を実装できた

⚠ つまづきポイント

問題	原因	解決策
自分のロールを変更できる	disabled 条件がない	<code>user.id === session?.user?.id</code> で無効化
変更が反映されない	キャッシュが残っている	<code>invalidate()</code> でキャッシュクリア
一般ユーザーがアクセスできる	権限チェックがない	<code>role !== 'ADMIN'</code> で早期リターン

次回予告

Day 25では、プロフィール編集機能を実装します。